

# TP3 BIS

Gambardello Clara

2025-03-08

## Partie A : Données d'électricité

Nous considérons les données de consommation électrique d'entreprises françaises mesurées toutes les 30 minutes pendant deux semaines par des compteurs Linky. L'objectif de cette partie est d'implémenter la méthode de validation croisée.

On commence par le télécharger et on regarde la taille du jeu de données d'électricité :

```
load("C:/Users/CLARA/OneDrive/Documents/M1/Semestre 8/Modélisation linéaire/TP/TP3 BIS/smart278co.Rdata")  
  
elec<-smart.278co  
dim(elec)
```

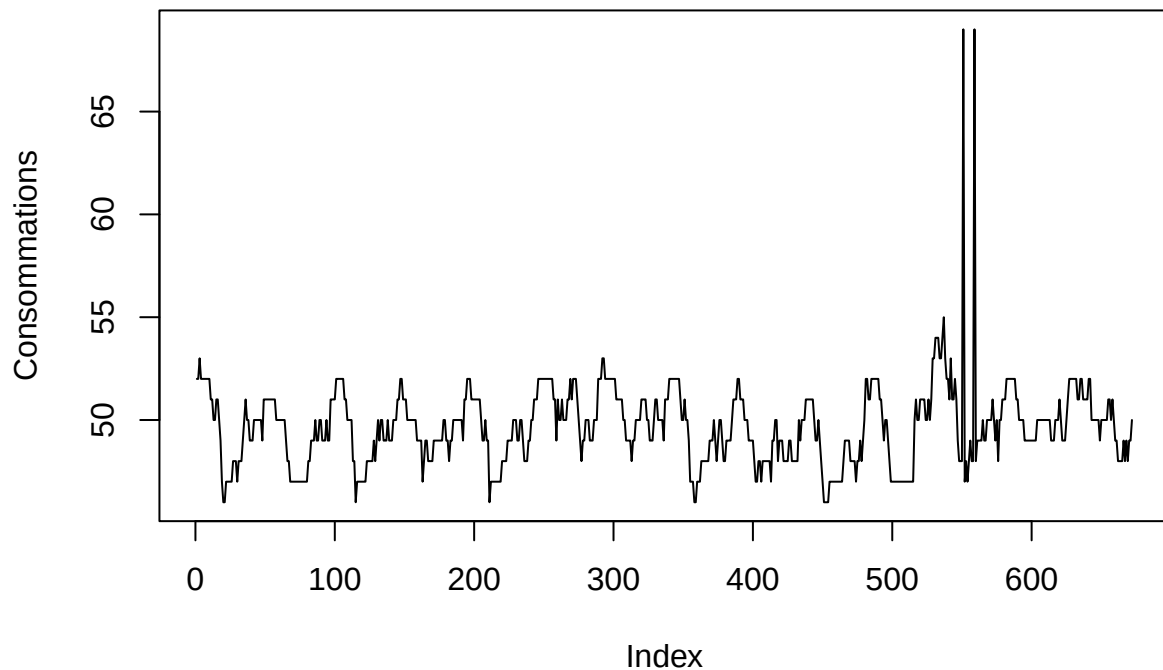
```
## [1] 6291 672
```

On voit que ce jeu de données comporte 672 variables, donc 672 mesures d'électricité et 6291 observations donc 6291 entreprises. On remarque dans l'environnement global, que la table "elec" est sous forme de matrice et non de data frame. Ce qui n'est pas plus mal car on a beaucoup d'observations et la manipulation de matrice est plus rapide que celle de data frame pour nos calculs "à la main".

On représente ensuite une courbe d'électricité pour visualiser les données d'une des entreprises :

```
plot(elec[100,], type = "l", main="Courbe d'électricité", ylab="Consommations")
```

## Courbe d'électricité



On remarque un pic de consommation entre 500 et 600.

On va tout d'abord extraire les variables explicatives X et la variable à expliquer y. La variable X est composée de la variable constante ainsi que des données enregistrées pendant la première semaine. La variable à expliquer y, représente la consommation électrique du lundi de la deuxième semaine.

Pour séparer le jeu de données en deux semaines on regarde tout d'abord le nombre de colonnes du jeu de données, divisé par deux pour avoir le nombre de variables explicatives que possède une semaine :

```
"Nombre de variable explicative par semaine :"
```

```
## [1] "Nombre de variable explicative par semaine :"
```

```
ncol(elec)/2
```

```
## [1] 336
```

On peut séparer le jeu de données en deux parties "semaine1" et "semaine2" qui comportent les deux 336 colonnes :

```
semaine1<-elec[,1:336]  
semaine2<-elec[,337:672]
```

On peut maintenant créer la matrice X qui contient toutes les variables de la table "semaine1", plus la variable constante :

```
X<-cbind(1, semaine1)
```

Il nous reste à extraire le vecteur  $y$ , pour cela on va repérer combien de relevés par jour sont faits (nombre de variables qui correspondent au lundi de la deuxième semaine).

le lundi de la deuxième semaine est donc à la colonne 1 du jeu de données “semaine2”. les données sont relevés toutes les 30 minutes donc dans 1 journée on a  $24 \times 2 = 48$  relevées donc 48 colonnes. Donc pour obtenir  $y$  on va faire la somme des lignes pour les colonnes 1 à 48.

```
y<-rowSums(semaine2[,1:48])
```

On a le modèle de régression multiple suivant :

$$(M_1) : y = X\beta + \epsilon$$

“elec” est une matrice et pour utiliser la fonction “lm” on a besoin d’une dataframe donc on transforme “X” en dataframe et on y ajoute “y”, on nomme cette data frame “data\_elec” et ce sera nos données sur lesquelles on effectuera la régression.

```
data_elec<-as.data.frame(cbind(y, X))
colnames(data_elec)[2]<- "Intercept"

n<-nrow(data_elec)
p<-ncol(data_elec)-1 #car la constante est dans data_elec
```

On aurait dû tracer un “pairplot” pour visualiser une potentielle tendance linéaire mais il y a beaucoup trop de variables pour le faire ici. On ajuste donc directement le modèle et on calcule les résidus d’estimation et les valeurs ajustées :

```
res<-lm(y~.-1, data=data_elec) # -1 car dans X il y'a déjà la constante
```

```
"Résidus d'estimations :"
```

```
## [1] "Résidus d'estimations :"
```

```
head(res$residuals)
```

```
##      X1000      X1001      X1002      X1003      X1004      X1005
## 1125.4623 -867.6965 1816.2767 6018.9495 7743.9258 3843.2930
```

```
"Valeurs ajustées :"
```

```
## [1] "Valeurs ajustées :"
```

```
head(res$fitted.values)
```

```
##      X1000      X1001      X1002      X1003      X1004      X1005
## 11846.538  7240.696 10280.723 27622.050 47528.074 24258.707
```

## Résidus de prévision calculés par validation croisée

**Méthode 1** On souhaite calculer les résidus de prévision calculés par validation croisée. Les résidus de prévision sont définis par :

$$\hat{\epsilon}_{(i)} = y_i - \hat{y}_{(i)}$$

pour tout  $i = 1, \dots, 6291$  où  $\hat{y}_{(i)}$  est la prédiction de  $y_i$  calculée à partir du modèle basé sur l'échantillon privé de l'observation  $i$ .

On considère le modèle linéaire privé de l'observation  $i$  :

$$(\mathcal{M}_{(i)}) : y_{(i)} = X_{(i)}\beta_{(i)} + \epsilon_{(i)}$$

L'estimateur  $\hat{\beta}_{(i)}$  dans ce modèle est :  $\hat{\beta}_{(i)} = (X_{(i)}^T X_{(i)})^{-1} X_{(i)}^T y_{(i)}$ . La prédiction de  $y_i$  basée sur le modèle  $(\mathcal{M}_{(i)})$  est  $\hat{y}_{(i)} = x_i^T \hat{\beta}_{(i)}$ .

Pour calculer les valeurs prédites, on utilise la fonction "predict" car  $\hat{y}_{(i)}$  est la prédiction de  $y_i$  calculée sur  $(\mathcal{M}_{(i)})$ .

```
res_prevision<-function(nb_ligne, data){
  res_pred_i<-rep(0,nb_ligne)
  for (i in 1:nb_ligne){
    data_elec_i<-data[-i,]
    res_i<-lm(y~.-1, data=data_elec_i)
    pred_i<-predict(res_i, newdata=data[i,])
    res_pred_i[i]<-data$y[i]-pred_i
  }
  return(res_pred_i)
}

system.time(res_pred_i<-res_prevision(n, data_elec))
```

```
## utilisateur      système      écoulé
##      3734.59      202.94      3945.28
```

On voit que cette fonction a un temps d'exécution énorme.

On obtient le vecteur  $\hat{\epsilon}_{(i)}$  suivant :

```
head(res_pred_i)
```

```
## [1] 1136.9120 -880.3589 1824.8148 6177.3755 8614.0847 3988.7481
```

**Méthode 2** Ici, on veut calculer  $\frac{\hat{\epsilon}_i}{(1-h_{ii})}$ , où  $\hat{\epsilon}_i$  est le résidu d'estimation de  $y_i$  à partir du modèle global. On sait d'après le cours que  $\hat{\epsilon}_{(i)} = \frac{\hat{\epsilon}_i}{(1-h_{ii})}$ , on devrait donc obtenir le même vecteur qu'à la question précédente. Cette méthode est beaucoup plus rapide car il n'y a pas de boucle et on n'a pas  $n$  modèles à calculer.

```
epsilon<-res$residuals
h_ii<-hatvalues(res)

"Temps d'execution : "
```

```
## [1] "Temps d'execution : "
```

```
system.time(epsilon_pred_i<-epsilon/(1-h_ii))
```

```
## utilisateur      système      écoulé  
##              0          0          0
```

```
"Résidus de prévision : "
```

```
## [1] "Résidus de prévision : "
```

```
head(epsilon_pred_i)
```

```
##      X1000      X1001      X1002      X1003      X1004      X1005  
## 1136.9120 -880.3589 1824.8148 6177.3755 8614.0847 3988.7481
```

On remarque que cette méthode est très rapide

On peut maintenant comparer le vecteur “res\_pred\_i” de la méthode 1 avec le vecteur “epsilon\_pred\_i” de la méthode 2 :

```
all(round(res_pred_i, digits=3)==round(epsilon_pred_i, digits=3))
```

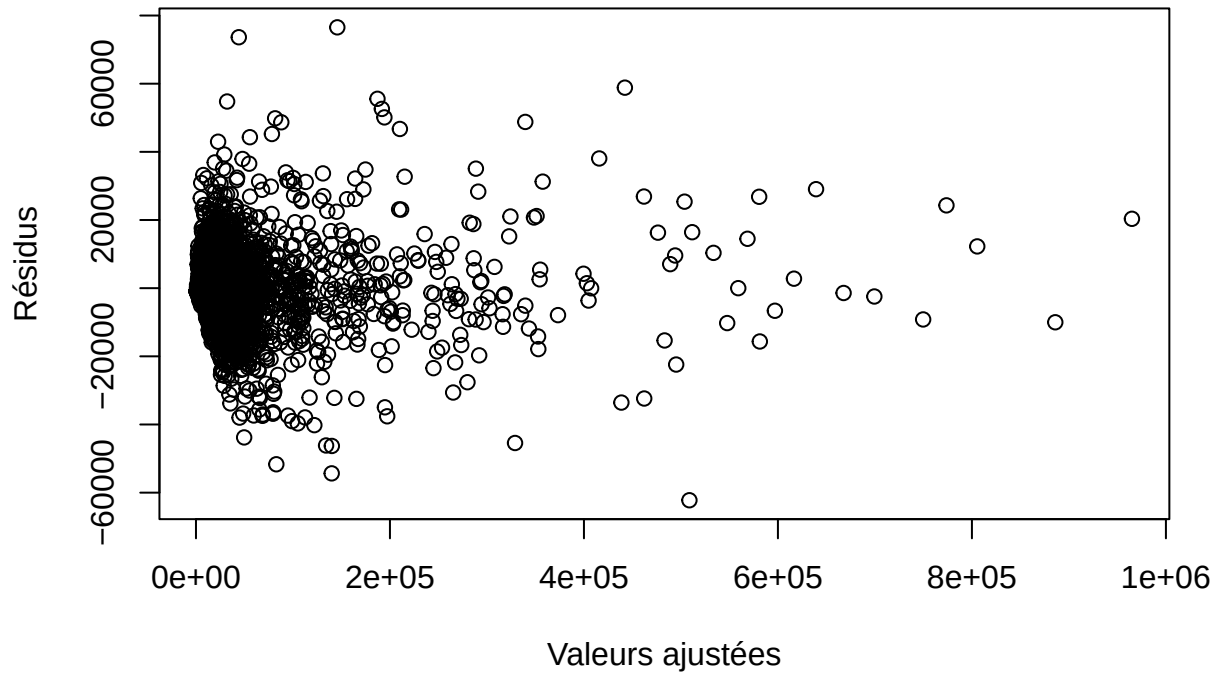
```
## [1] TRUE
```

On remarque que les deux méthodes sont équivalentes car les deux vecteurs sont égaux. Donc on a bien  $\hat{\epsilon}_{(i)} = y_i - \hat{y}_{(i)} = \frac{\epsilon_i}{(1-h_{ii})}$

En tant que futur data scientist, je dirais que cette modélisation est un bon début pour voir les relations simples entre les variables mais que le modèle n’est pas forcément très pertinent. Par exemple, si on trace les résidus d’estimation en fonction des valeurs ajustées on obtient le graphique suivant :

```
plot(res$fitted.values, res$residuals, main="Résidus en fonction des valeurs ajustées",  
      xlab="Valeurs ajustées", ylab="Résidus")
```

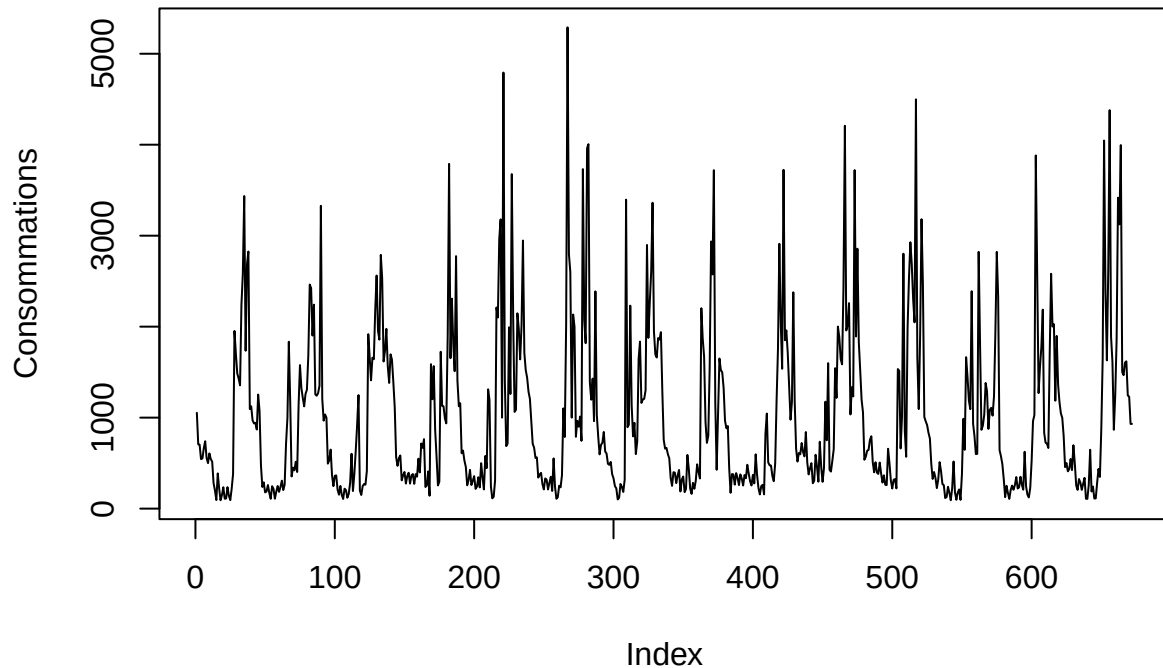
## Résidus en fonction des valeurs ajustées



On remarque une tendance, la plupart des observations sont regroupées vers la gauche, ce qui peut faire penser que le modèle n'est pas pertinent. De plus, si on trace une nouvelle courbe d'électricité pour une autre entreprise que celle calculée précédemment, on remarque que les entreprises peuvent avoir des habitudes de consommations différentes ce qui peut créer des valeurs aberrantes dans les données et les modèles linéaire y sont sensible.

```
plot(elec[10,], type = "l", main="Courbe d'électricité", ylab="Consommations")
```

## Courbe d'électricité



Il faudrait donc essayer d'autres méthodes et voir laquelle serait la plus adaptée.

## Partie B : programmer à la main

On commence par construire la matrice "hat", qui est la matrice de projection définie par :

$$P_X = X(X^T X)^{-1} X^T = (h_{ii})_{i=1}^n$$

. Pour calculer  $P_X$  on prend comme matrice  $X$ , la matrice définie plus haut composée de la variable constante et des autres variables explicatives. Pour que le calcul soit plus rapide on utilise les fonctions "solve(X)" et "crossprod(X,X)" qui respectivement, inverse une matrice et calcule  $X^T X$ .

```
system.time({
Px<-X%*%solve(crossprod(X))%*%t(X)
h<-diag(Px)
})
```

```
## utilisateur      système      écoulé
##           9.37         0.13         9.61
```

```
#Vérification avec la fonction de R :
system.time(h_R<-hatvalues(res))
```

```
## utilisateur      système      écoulé
##           0.81         0.00         0.81
```

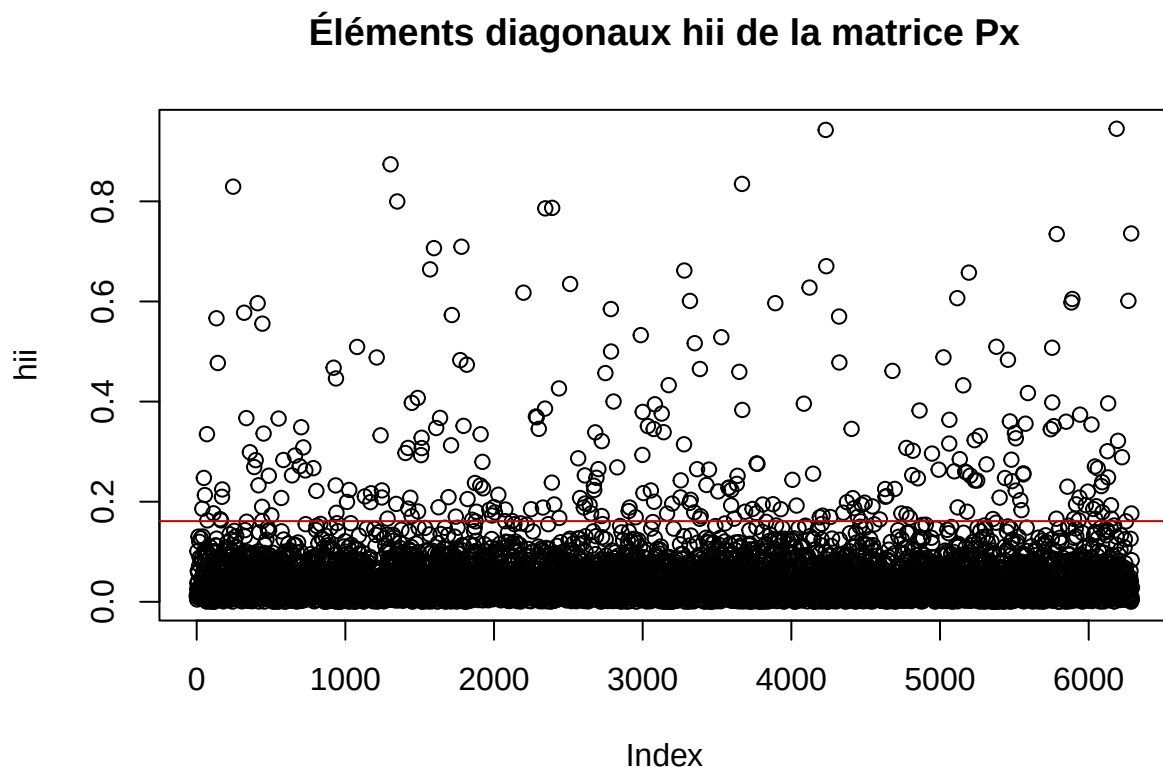
```
all(round(h, digits=4)==round(h_R, digits=4))
```

```
## [1] TRUE
```

Les deux vecteurs sont bien égaux. la manière la plus rapide est de calculer avec “lm” car à la main le fait de calculer “Px” est plus long à cause des produits matriciels.

On représente graphiquement les  $h_{ii}$ , et on trace le seuil :

```
plot(h, main="Éléments diagonaux hii de la matrice Px", ylab="hii")
abline(h=3*(p+1)/n, col="red3")
```



Ces quantités sont utiles dans l’analyse des points leviers. On voit ici que la plupart des points sont en dessous du seuil mais qu’il y a quand même beaucoup de points au dessus du seuil, il y’a donc beaucoup de points leviers.

On souhaite désormais, à partir de  $P_X$ , déduire le vecteur des valeurs ajustées  $\hat{y}$ , et le vecteur des résidus d’estimation  $\hat{\epsilon} = y - \hat{y}$ .

On a  $\hat{y} = P_X \times y$ .

```
system.time({
  hat_y<-Px%*%y
  hat_epsilon<-y-hat_y})
```

```
## utilisateur      système      écoulé
##          0.06         0.00         0.07
```

```
#Vérification avec la fonction de R :
system.time(hat_y_R<-res$fitted.values)
```

```
## utilisateur      système      écoulé
##           0           0           0
```

```
system.time(hat_epsilon_R<-res$residuals)
```

```
## utilisateur      système      écoulé
##           0           0           0
```

```
all(round(hat_y, digits=3)==round(hat_y_R, digits=3))
```

```
## [1] TRUE
```

```
all(round(hat_epsilon, digits=3)==round(hat_epsilon_R, digits=3))
```

```
## [1] TRUE
```

Les deux vecteurs des valeurs ajustées sont égaux ainsi que ceux pour les résidus d'estimation. Les deux façons de faire sont rapide, avec un léger gain de temps pour celle avec la fonction "lm".

### Résidus par validation croisée

Dans la partie A on a vu que  $\hat{\epsilon}_{(i)} = y_i - \hat{y}_{(i)} = \frac{\hat{\epsilon}_i}{(1-h_{ii})}$ , et donc en utilisant la formule  $\hat{\epsilon}_{(i)} = \frac{\hat{\epsilon}_i}{(1-h_{ii})}$ , on peut calculer les résidus par validation croisée plus rapidement et plus simplement car on a pas besoin construire n modèles pour obtenir les erreurs de prévision.

Pour les calculer, on va simplement utiliser le vecteur des résidus d'estimation "hat\_epsilon" et h\_ii calculés plus haut :

```
epsilon_i<-hat_epsilon/(1-h)
head(epsilon_i)
```

```
##           [,1]
## X1000 1136.9120
## X1001 -880.3589
## X1002 1824.8148
## X1003 6177.3755
## X1004 8614.0847
## X1005 3988.7481
```

### Distance de cook

On va maintenant donner une implémentation simple pour calculer les distances de Cook  $C_i$  pour  $i = 1, \dots, 6291$ . La distance de cook mesure l'influence de l'observation  $i$  sur l'estimation du paramètre  $\beta$ . Elle est définie comme suit :

$$C_i = \frac{1}{(p+1)\hat{\sigma}^2} (\hat{\beta}_{(i)} - \hat{\beta})^T X^T X (\hat{\beta}_{(i)} - \hat{\beta})$$

. Avec cette définition on doit calculer  $n$  modèles ce qui est très coûteux en calcul dans un algorithme, on va donc ici implémenter la distance de cook en utilisant un résultat du cours qui dit que l'on peut également écrire la distance de cook comme ci-dessous :

$$C_i = \frac{1}{(p+1)} \frac{h_{ii}}{(1-h_{ii})} t_i^2 = \frac{1}{(p+1)} \frac{h_{ii}}{(1-h_{ii})^2} \frac{\hat{\epsilon}_i^2}{\hat{\sigma}^2}$$

où,  $t_i$  sont les résidus standardisés, et  $\hat{\sigma}^2$  l'estimateur sans biais de la variance définit par  $\frac{\|\hat{\epsilon}\|^2}{n-(p+1)}$ .

```
cook<-function(hii, nb_row, nb_col, hat_epsilon){
  sigma2<-sum(hat_epsilon^2)/(nb_row-(nb_col+1))
  C_i<-((hat_epsilon^2)/((nb_col+1)*sigma2))*(hii/(1-hii)^2)
  return(C_i)
}
```

On peut maintenant calculer la distance de cook avec notre fonction et avec la fonction de R : “cooks.distance” qui prend en argument le résultat de la fonction “lm”. Puis on fait un test d'égalité des deux vecteurs obtenus :

```
system.time(D_cook<-cook(h, n, p, hat_epsilon))
```

```
## utilisateur      système      écoulé
##              0              0              0
```

```
#Vérification avec la fonction de R :
system.time(D_cook_R<-cooks.distance(res))
```

```
## utilisateur      système      écoulé
##           0.85           0.00           0.85
```

```
all(round(D_cook, digits=1)==round(D_cook_R, digits=1))
```

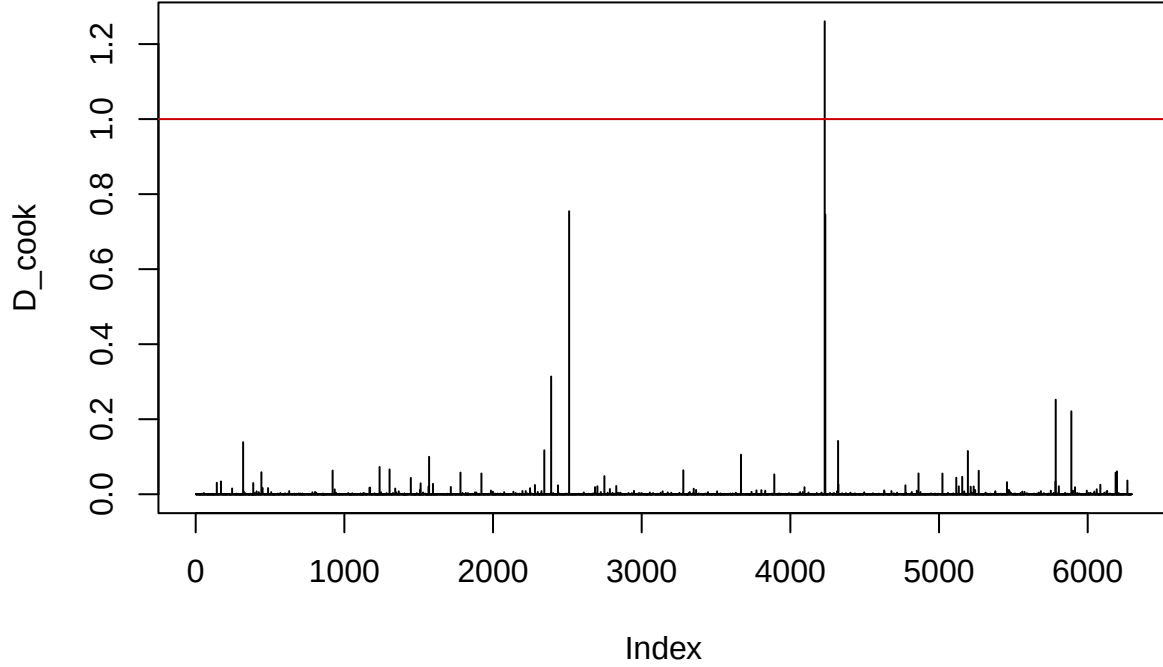
```
## [1] TRUE
```

On voit bien que notre fonction et la fonction de R renvoient la même chose. De plus, notre fonction est légèrement plus rapide que celle de R.

On trace ensuite le plot des distances de cook ainsi que le seuil qu'on place à 1 (bonne approximation de  $f_{p+1,n-(p+1)}(0.5)$ ) :

```
plot(D_cook, type="h", main="Distance de Cook")
abline(1, 0, col="red3")
```

## Distance de Cook



Quelques observations ont une distance de Cook supérieure aux autres observations, comme le pic vers 2500 ou vers 4200 environ. De plus seulement 1 observation dépasse le seuil, donc cette observations à une forte influence sur le modèle.

## Résidus studentisés par validation croisée

On souhaite maintenant calculer les résidus studentisés par validation croisée  $t_i^*$ .

$$t_i^* = \frac{\hat{\epsilon}_i}{\hat{\sigma}_{(i)} \sqrt{1 - h_{ii}}}$$

. Pour cela, on doit calculer  $\hat{\sigma}_{(i)}$  qui est défini par :

$$\hat{\sigma}_{(i)} = \frac{\sum_{j=1, j \neq i}^n \hat{\epsilon}_{j(i)}^2}{n - 1 - (p + 1)}$$

où  $\hat{\epsilon}_{j(i)} = y_j - y_{j(i)}^{\hat{}}$  avec  $y_{j(i)}^{\hat{}} = x_j^T \hat{\beta}_{(i)}$ .

Pour éviter de calculer n modèles, on va exprimer  $\hat{\sigma}_{(i)}$  par rapport au modèle construit sur l'échantillon total de n observations. On va donc montrer qu'on peut exprimer les résidus  $\hat{\epsilon}_{j(i)}$  en fonction des résidus  $\hat{\epsilon}_i$ .

Démonstration :

Tout d'abord on a  $\hat{\epsilon}_{j(i)} = y_j - y_{j(i)}^{\hat{}}$  or,  $y_{j(i)}^{\hat{}} = x_j^T \hat{\beta}_{(i)}$  et on a également  $\hat{\beta}_{(i)} = (X_{(i)}^T X_{(i)})^{-1} X_{(i)}^T y_{(i)}$ . D'après l'exercice du TD on a les expressions suivante :  $X_{(i)}^T X_{(i)} = X^T X - x_i x_i^T$  et  $X_{(i)}^T y_{(i)} = X^T y - x_i y_i$ .

On va utiliser le résultat suivant :  $(M + uv^T)^{-1} = M^{-1} - \frac{M^{-1}uv^T M^{-1}}{1 + v^T M^{-1}u}$ , ou M est une matrice symétrique et inversible.

Comme  $X_{(i)}^T X_{(i)}$  est symétrique et inversible, on peut lui appliquer le résultat précédent et on obtient :  $(X_{(i)}^T X_{(i)})^{-1} = (X^T X)^{-1} + \frac{1}{1-h_{ii}}(X^T X)^{-1} x_i x_i^T (X^T X)^{-1}$ .

On en déduit ensuite  $\hat{\beta}_{(i)}$  :

$$\hat{\beta}_{(i)} = (X_{(i)}^T X_{(i)})^{-1} X_{(i)}^T y_{(i)} = [(X^T X)^{-1} + \frac{1}{1-h_{ii}}(X^T X)^{-1} x_i x_i^T (X^T X)^{-1}] (X^T y - x_i y_i) = \hat{\beta} + \frac{1}{1-h_{ii}}(X^T X)^{-1} x_i \hat{y}_i - (X^T X)^{-1} x_i y_i - \frac{1}{1-h_{ii}}(X^T X)^{-1} x_i h_{ii} y_i.$$

En développant, on a remplacer :  $x_i^T \hat{\beta} = \hat{y}_i$ ,  $(X^T X)^{-1} X^T y = \hat{\beta}$  et  $x_i^T (X^T X)^{-1} x_i = h_{ii}$ .

On peut maintenant calculer  $\hat{y}_{j(i)}$  :

$$\hat{y}_{j(i)} = x_j^T \hat{\beta}_{(i)} = x_j^T \hat{\beta} + \frac{1}{1-h_{ii}} x_j^T (X^T X)^{-1} x_i \hat{y}_i - x_j^T (X^T X)^{-1} x_i y_i - \frac{1}{1-h_{ii}} x_j^T (X^T X)^{-1} x_i h_{ii} y_i$$

$$\text{or, } x_j^T (X^T X)^{-1} x_i = h_{ij} \text{ donc } \hat{y}_{j(i)} = \hat{y}_j - h_{ij} y_i + \frac{h_{ij}}{1-h_{ii}} (\hat{y}_i - h_{ii} y_i).$$

$$\text{Il ne reste plus qu'à remplacer dans } \hat{\epsilon}_{j(i)} : \hat{\epsilon}_{j(i)} = y_j - \hat{y}_{j(i)} = y_j - \hat{y}_j + h_{ij} y_i - \frac{h_{ij}}{1-h_{ii}} (\hat{y}_i - h_{ii} y_i) = \hat{\epsilon}_j + \frac{h_{ij}}{1-h_{ii}} (-\hat{y}_i - h_{ii} y_i + \frac{h_{ij}(1-h_{ii})y_i}{h_{ij}}) = \hat{\epsilon}_j + \frac{h_{ij}}{1-h_{ii}} (y_i - \hat{y}_i) = \hat{\epsilon}_j + \frac{h_{ij}}{1-h_{ii}} \hat{\epsilon}_i \text{ pour } j \neq i.$$

$$\text{et pour } j = i, \text{ on a : } \hat{\epsilon}_{i(i)} = \hat{\epsilon}_i + \frac{h_{ii}}{1-h_{ii}} \hat{\epsilon}_i = \hat{\epsilon}_i \left( \frac{1-h_{ii}+h_{ii}}{1-h_{ii}} \right) = \frac{1}{1-h_{ii}} \hat{\epsilon}_i.$$

On va maintenant implémenter la fonction pour calculer  $\hat{\sigma}_{(i)}$  :

```
sigma_i_esti<-function(n, p, hat_epsilon, Px, hii){
  sigma_i<-rep(0,n)
  for (i in 1:n) {
    epsilon_j_i<-hat_epsilon[-i]+(Px[i,-i]/(1-hii[i]))*hat_epsilon[i]
    sigma_i[i]<-sum(epsilon_j_i^2)/(n-1-(p+1))
  }
  return(sigma_i)
}

sigma_i<-sigma_i_esti(n, p, hat_epsilon, Px, h)
head(sigma_i)
```

```
## [1] 72351930 72352017 72351588 72345898 72340938 72349570
```

On peut maintenant calculer les résidus studentisés par validation croisée, puis comparer le vecteur obtenu a celui obtenu avec la fonction “rstudent” :

```
res_studentise<-function(sigma_i, hii, hat_epsilon){
  t_i_star<-rep(0,n)
  t_i_star<-hat_epsilon/(((sigma_i)^(1/2))*((1-hii)^(1/2)))
  return(t_i_star)
}

system.time(t_star<-res_studentise(sigma_i, h, hat_epsilon))
```

```
## utilisateur      système      écoulé
##              0              0              0
```

```
#Vérification avec la fonction de R :
system.time(t_star_R<-rstudent(res))
```

```
## utilisateur      système      écoulé  
##           0.86         0.01         0.86
```

```
all(round(t_star)==round(t_star_R))
```

```
## [1] TRUE
```

On obtient bien le même résultat. Encore une fois ici notre fonction est légèrement plus rapide que la fonction “rstudent”.